

U

O

W

CSIT314

Software Development

Methodologies

Principles and practices of continuous integration and delivery
(CI/CD)



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Continuous Integration (CI)



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

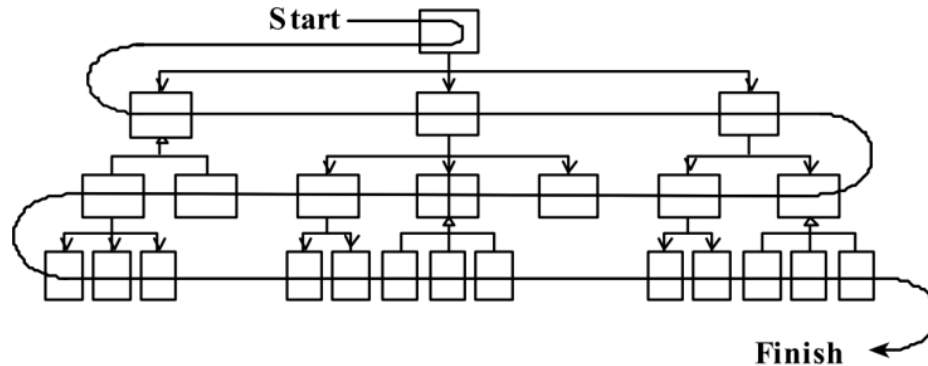
What is integration

- Multiple developers working on the same codebase at the same time (independently):
 - Developer A works on feature 1 while developer B works on feature 2.
 - Developer A works on class 123.java while developer B works on class 456.java
 - Once they have finished, they need to integrate their work into the main codebase.



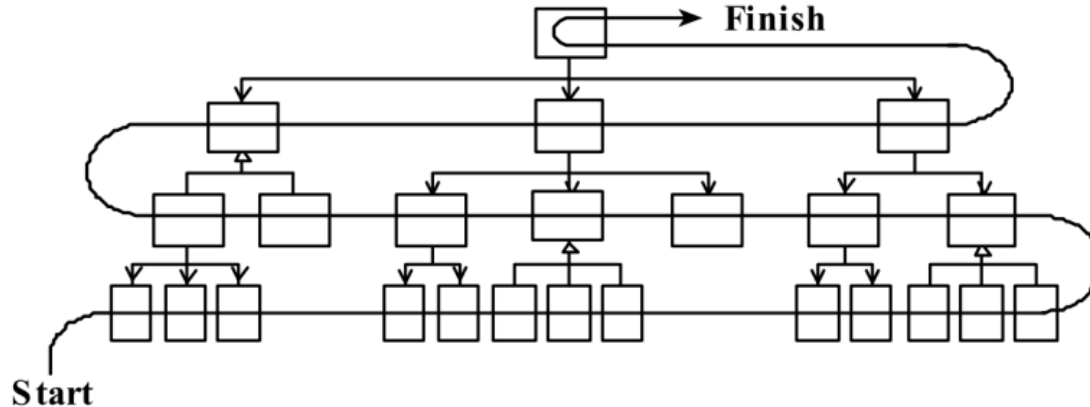
Top-down integration

- Start with top-level UI layers
 - must write (lots of) stub lower layers for UI to interact with
 - allows postponing tough design/debugging decisions



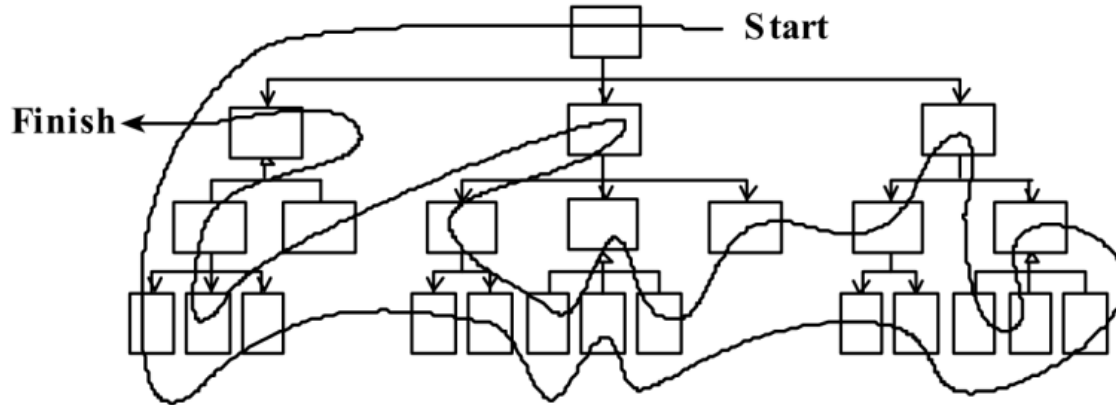
Bottom-up integration

- Start with bottom-level data/logic layers
 - must write test drivers to run these layers
 - won't discover high-level / UI design flaws until late



Sandwich integration

- Combine top-level UI with bottom-level classes
 - add middle layers later as needed
 - more flexible but also problems



Continuous Integration (CI)

- Continuous integration (CI) is a software development practice where developers in a team integrate their work frequently.
 - several times a day
- Each integration can be verified by an automated build:
 - Compile (or build) the code
 - translating human-readable source code into machine-readable instructions, typically performed by a compiler
 - Run automated tests



Build Types

- There are three basic types of builds: private, integration, and release (more on next slide)
 - All builds are automated, script-driven processes that pull source code from the repository and create deployable artifacts
 - These build types differ in several ways
 - Where, when, by whom, and for what reason they are run
 - The code that feeds the process
 - What is done with the output



Build Types

- Private Builds
 - Run by developers in their local environments (IDE)
 - Ensure that code compiles before committing to source code repository
 - Triggered manually
 - Very stripped-down and designed to finish quickly
- Integration Builds
 - Compile in response to recent changes
 - Performed on a dedicated build server either on a scheduled basis (nightly) or continuously
 - Usually drawn from the trunk of the repository tree
 - Fast or slow depending on the number and type of processes included in the build script
- Release Builds
 - Performed codes to production environments
 - Performed by the team' s release manager
 - Run the full suite of automated and manual tests
 - Tend to be slow



Build from command line (example)

- An Android project needs a build.xml to be used by Ant.
 - This file allows your project to be compiled from the command line, making automated builds possible.

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="MainActivity" default="help">
  <property file="ant.properties" />
  <import file="${sdk.dir}/tools/ant/build.xml" />
  <taskdef name="findbugs" classname="edu.umd.cs.findbugs.anttask.FindBugsTask"/>
  <target name="findbugs">
    <findbugs home="${findbugs.home}" output="xml" outputFile="findbugs.xml"
      excludeFilter="findbugs-exclude.xml">
      <auxClasspath path="${android.jar}" />
      <auxClasspath path="${rt.jar}" />
      <auxClasspath path="libs\android-support-v4.jar" />
      <class location="${out.dir}" />
    </findbugs>
  </target>

  <taskdef resource="checkstyletask.properties" classpath="${basedir}/libs/checkstyle-5.6-all.jar"/>
  <checkstyle config="sun_checks.xml" failonviolation="false">
    <fileset dir="src" includes="**/*.java"/>
    <formatter type="plain"/>
    <formatter type="xml" toFile="checkstyle-result.xml"/>
  </checkstyle>
</project>
```

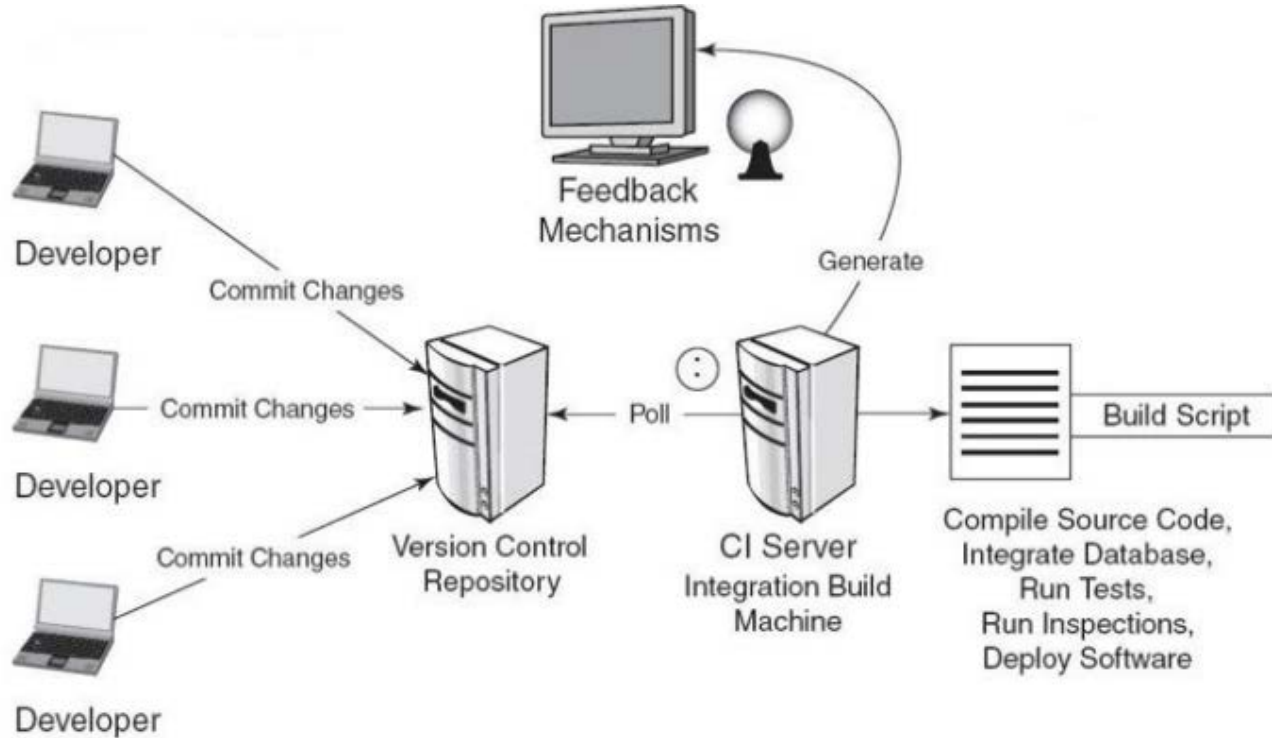


Daily builds

- Daily build: Compile working executable on a daily basis
 - allow test the quality of your integration
 - make sure product “works every day”
 - through an easy script
 - quickly catches/exposes bug that breaks the build
- Server: An external machine that automatically pulls down your latest repo code and fully builds all resources.
 - If anything fails, contacts the team
 - Ensures that the build is never broken for long



Architecture



Artifacts Needed

- A build usually involves more than source code files. Other items may include:
 - Project and third-party components and libraries
 - Configuration files
 - Data files and database scripts
 - Test scripts
 - Properties files
- All items except project and third-party libraries should be stored in the source code repository.



Build System

- completely automated
- usually through the execution of a single script
- a build always begins by retrieving code from the source code repository
- unless terminated prematurely, the end product of a build is always executable code
- notification of build status always occurs through a feedback mechanism



The build cycle

- Basic build cycle consists of these key steps:
 - Developer commits changes to the source code repository
 - Build server executes the master build script, or delegates execution to another server
 - Checks out source code
 - Builds executable version of the application
 - Runs other jobs, such as testing and code inspection
 - Team is notified of build results through a feedback mechanism
 - If alerts are generated, the team takes immediate action to correct problems
 - If a code fix is needed, developer commits the corrected code back to the repository; this action kicks off a new build cycle.



Improving Build Speed

- Three approaches to dealing with build speed:
 - Speed up the build by eliminating bottlenecks or by running the build on a faster machine with plenty of memory.
 - Pick a good time: run complete builds less often or at a time when developers are less likely to be working
 - Employ a “staged/seperate build” approach: the basic build, the 2nd build for another process, the 3rd build for remaining steps.
- A combination of the three approaches is often used.

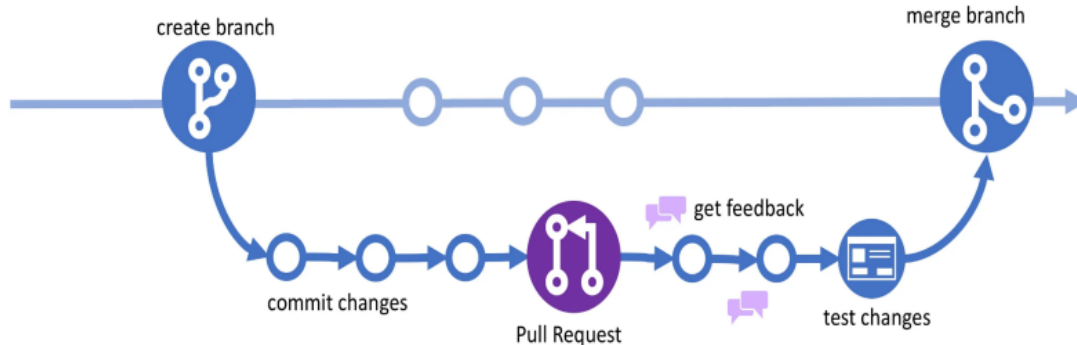


Continuous Integration

- Principles (detail later)
 - maintain a single source repository
 - automate the build
 - make your build self-testing (Automated tests)
 - everyone commits to mainline every day
 - every commit should build mainline on an integration machine
 - keep the build fast
 - test in a clone of the production environment
 - make it easy for anyone to get the latest executable
 - everyone can see what's happening

Practices of Continuous Integration

- Maintain a Single Source Repository
 - Use a version control system such as Git/GitHub
 - **Everything** required to build the software app should be in the repository (code, test scripts, test data, properties files, database schema, third-party libraries, etc.)



Practices of Continuous Integration

- Automate the build
 - Automate the whole process of turning the source code into a running software app
 - Operations: compiling, moving files around, loading schemas into the databases, etc.
 - Issuing a single command and the whole build process will run automatically.



Practices of Continuous Integration

- Make your build self-testing
 - automated tests: e.g. Tests that can be run from the command line on your project code at any time.
 - can be unit tests, coverage, static analysis (inspection)
 - an important component of the CI build cycle (more later)
 - smoke test: A quick set of tests run on the daily build.
 - NOT exhaustive; just sees whether code "smokes" (breaks)
 - run with daily build



Practices of Continuous Integration

- Daily commits
- Submit work to main repo at end of each day.
 - Merge conflicts
 - Maintain the daily commit practice
 - If your code is not ready to submit at end of day, either submit a subset or be flexible about commit schedule



Practices of Continuous Integration

- Every commit should build the mainline (the main codebase) on an integration server/system
 - use continuous integration server/system (e.g. Jenkins, Travis CI, Bamboo, GitLab, etc.)
 - only if this integration build succeeds should the commit be considered to be done
- Test in a clone of the production environment.
 - Testing in a different environment introduces risk when the system is deployed in production.



Practices of Continuous Integration

- Commits should build the mainline (the main codebase) on an integration machine
 - Can be multiple times a day
 - Frequent integration means frequent communication
 - Find out if there is a conflict between the changes
 - Errors, conflicts, bugs, etc. can be detected early and rectified (fixed) quickly.



Practices of Continuous Integration

- Keep the build fast
 - to provide rapid feedback
 - if the build is not fast, developers will commit less often and will be provided feedback on problems less often.
 - also, long builds can be a significant roadblock to wait
- In general, the more processes included in the build, the better the quality of the final product
- However, there are tradeoffs
 - sub-process takes time to execute
 - extra steps result in loss of productivity, as developers wait for build cycles before they continue working



Continuous Database Integration

- Changes to a **database** may occur often
 - as important as early detection of errors in the source code.
 - a build server can be set up to begin a new CI cycle whenever database is changed.
 - code and configuration files that modify the database need also be committed to the repository
 - just as code changes



Infrastructure as Code

- What is IT infrastructure?
 - Physical machines, devices, OS, databases, and any other systems that are used to run a software application.
- Infrastructure as Code
 - the whole IT infrastructure can be treated as if they are software
 - setup, configured and changed automatically through writing code
- Infrastructure as code is the prerequisite for common CI/CD practices



Infrastructure as Code

- IaC model generates the same environment every time it is applied
 - enables software teams to test applications in production-like environments early in the development cycle
 - IaC demo: https://youtu.be/k6_ZTlxI4xk



Continuous Database Integration

- A CI build can:
 - Drop an existing database
 - Inspect database scripts for compliance with project practice
 - Create, configure, and populate data
 - Run automated unit tests to verify that everything is working as expected



Benefits of CI

- Increased productivity
 - Enables shorter feedback cycle when changes are made
 - Code is kept in a “releasable” state
 - Code gets back into the hands of testers quickly
 - Enables more frequent releases with new features
- Improved quality
 - Frequent testing identifies bugs
 - Multi-platform builds help in finding problems
- Reduced Risk
 - Reduces uncertainty greatly because at all times the team knows what works, what does not, and what the major issues are.



Barriers to CI

- Complexity and tradeoff
 - Time
 - Technical knowledge
 - A number of new tools and processes must be mastered
 - Setting up a CI server
 - Automated test
 - A build scripting language and potentially a setup/install platform as well.



Continuous Delivery (CD)



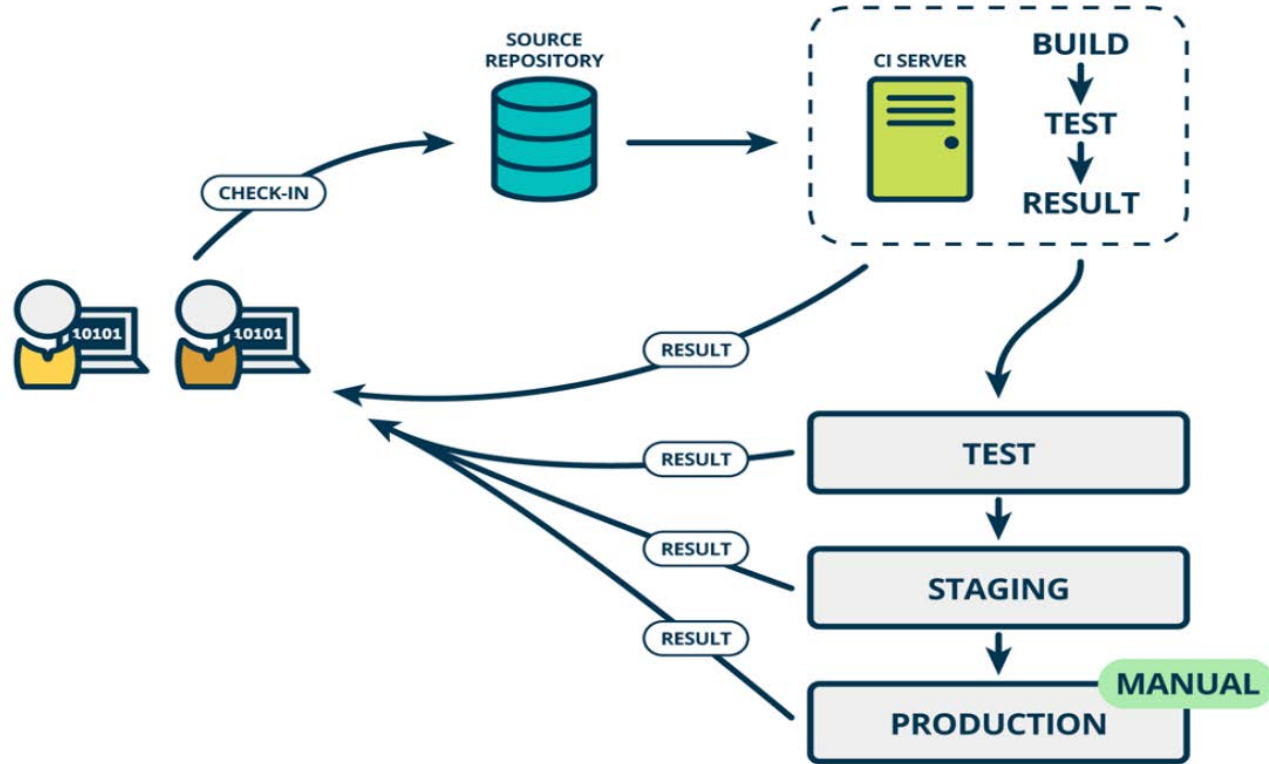
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Continuous Delivery

- Continuous Delivery is a further step beyond Continuous Integration:
 - Each time changes are pushed to the codebase, the new code is automatically built and tested on environments that are very similar to production (staging environment)
- The staging environment can be used to address non-functional requirements
 - such as security, redundancy, and flexibility
 - note that these may not be covered in the development environments



Continuous Delivery cycle

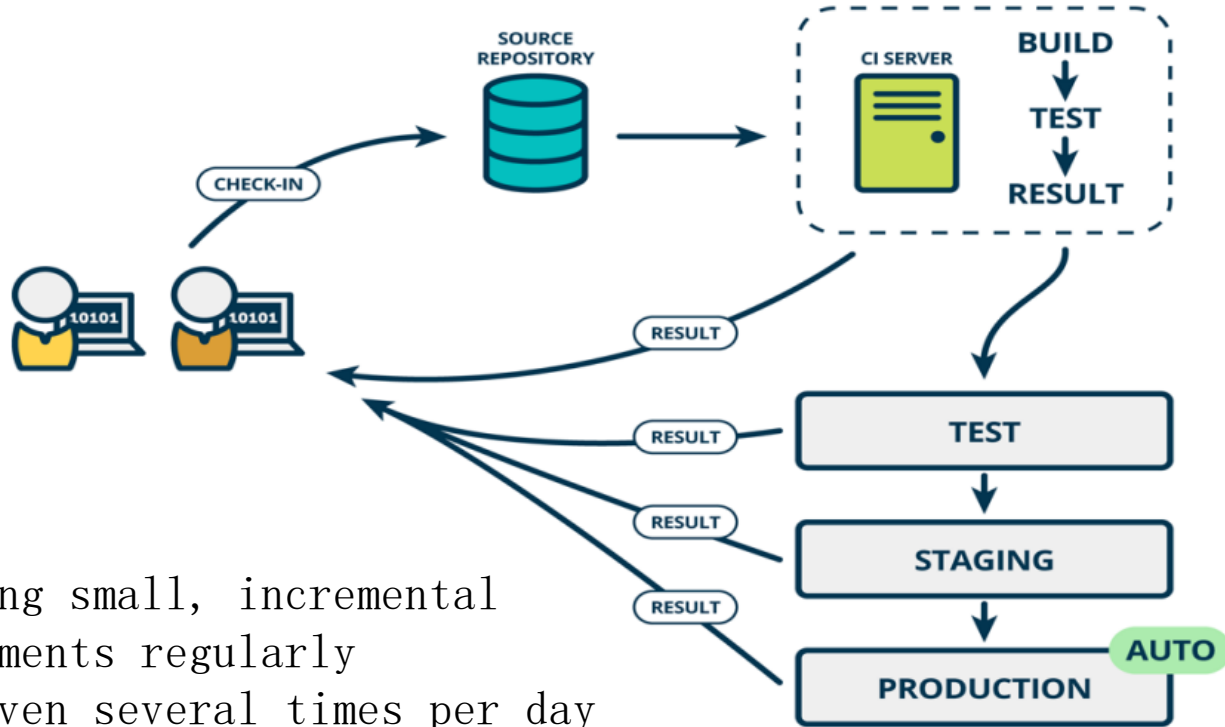


Continuous Deployment

- In Continuous Delivery, the deployment of the software application to a production environment is still done manually.
 - such as the manual testing
- Continuous Deployment is an additional step beyond Continuous Delivery (prototypes):
 - The software application is deployed automatically.
 - Every time code changes are pushed to the codebase, it will be automatically built and tested - and if the tests are successfully, it will automatically go to production.



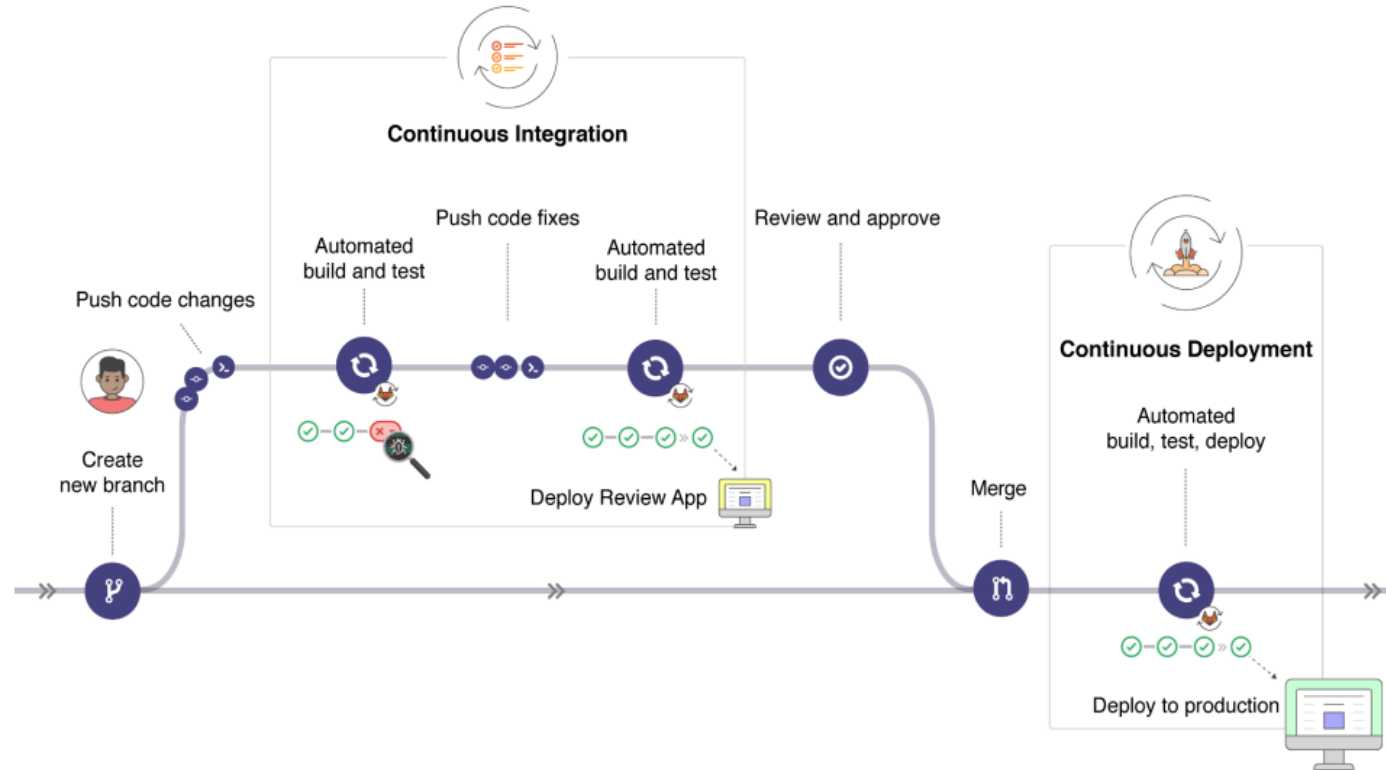
Continuous Deployment cycle



1. releasing small, incremental improvements regularly
2. often even several times per day



CI/CD with GitLab



CI/CD with GitLab (cont.)

- Setup a project on GitLab
- Install GitLab Runner on your computer
 - <https://docs.gitlab.com/runner/#install-gitlab-runner>
- Register GitLab Runner with GitLab
 - <https://docs.gitlab.com/runner/register/>
- Start GitLab Runner



CI/CD with GitLab (cont.)

- Create a CI/CD pipeline on GitLab
 - Create/add a file called “.gitlab-ci.yml” in the root folder of your project.
 - This YAML file defines the structure and order of the CI/CD pipeline and decides:
 - What will be executed using GitLab Runner
 - What to do when some specific conditions are satisfied (e.g. when a process succeeds or fails).
 - See <https://docs.gitlab.com/ee/ci/yaml/> for more details



CI/CD with GitLab (cont.)

- A live demo of how to set up and run CI/CD with GitLab
 - [https://www.youtube.com/watch?v=jUiKi6FWYrg&list=PLhW3qG5bs-L8YSnCiyQ-jD8XfHC2W1NL &index=8](https://www.youtube.com/watch?v=jUiKi6FWYrg&list=PLhW3qG5bs-L8YSnCiyQ-jD8XfHC2W1NL&index=8)
- GitLab CI/CD Examples for different types of applications written in different programming languages
 - <https://docs.gitlab.com/ee/ci/examples/>

